

06 — Schema Design: Banking System

Table of Contents

- 1. [Learning Objectives](#)
- 2. [Domain Overview](#)
- 3. [ASCII ER Diagram](#)
- 4. [The Double-Entry Ledger Model](#)
- 5. [Complete DDL Script](#)
- 6. [Key Design Decisions](#)
- 7. [Common Queries](#)
- 8. [ACID Guarantees in Banking](#)
- 9. [Common Mistakes](#)
- 10. [Best Practices](#)
- 11. [Performance Considerations](#)
- 12. [Interview Questions & Answers](#)
- 13. [Exercises with Solutions](#)
- 14. [Cross-References](#)

Learning Objectives

After completing this section you will be able to:

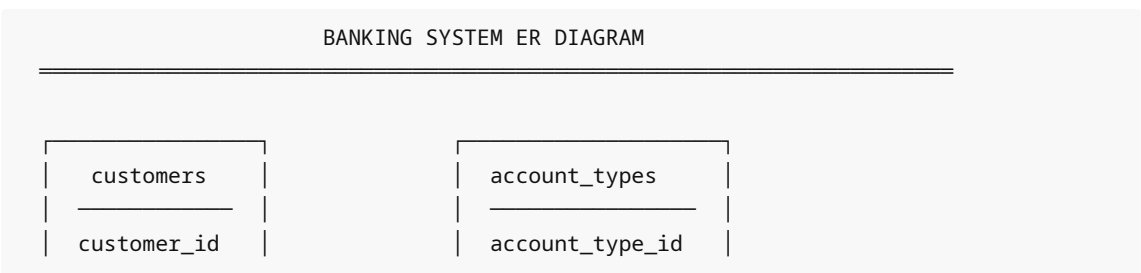
- Design a banking schema using the double-entry ledger model
- Ensure atomicity and consistency for fund transfers
- Handle account types, balance calculations, and reconciliation
- Understand why running totals are dangerous and how to compute balances safely
- Design audit-friendly financial schemas

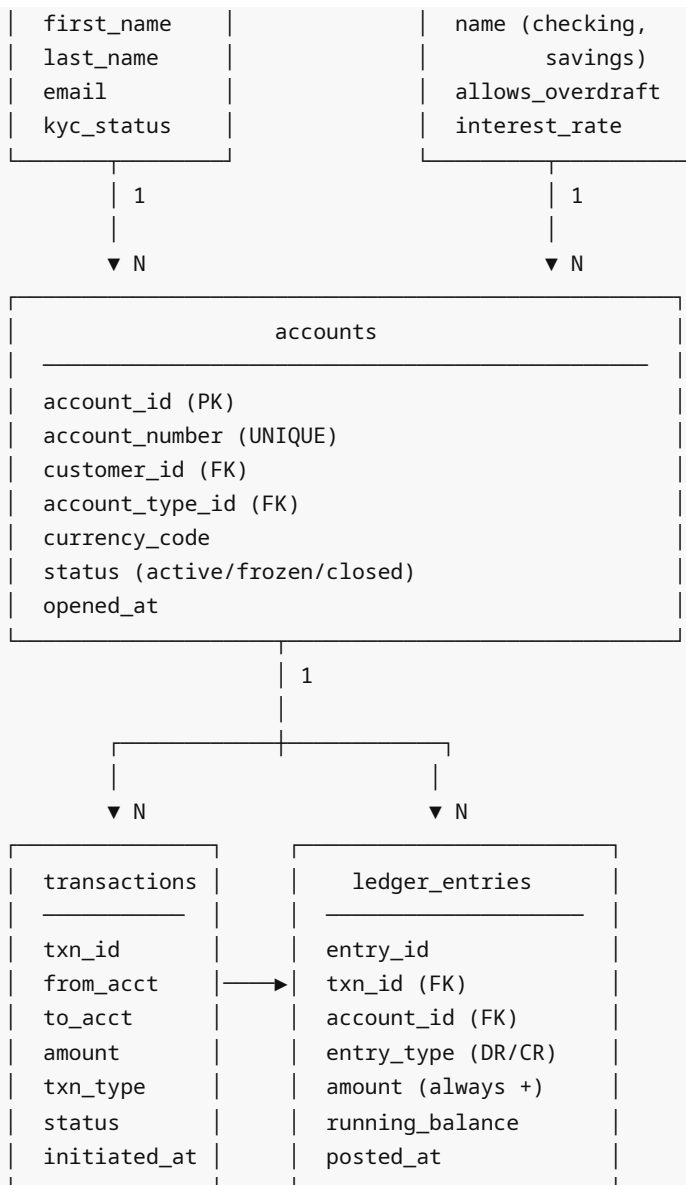
Domain Overview

A banking system manages:

- **Customers:** KYC data, identity verification
- **Accounts:** Checking, savings, credit accounts
- **Ledger entries:** Double-entry bookkeeping (every transaction is two entries)
- **Transactions:** High-level transfer records
- **Balance calculations:** Always derived from ledger, never stored independently
- **Compliance:** Audit log, regulatory reporting

ASCII ER Diagram





Business rule (double-entry): every transaction generates exactly:

- 1 DEBIT entry on the source account
- 1 CREDIT entry on the destination account

$\text{sum}(\text{DEBIT amounts}) = \text{sum}(\text{CREDIT amounts})$ for every transaction

The Double-Entry Ledger Model

Why double-entry?

Double-entry accounting is a 500-year-old principle that ensures every financial event is recorded in at least two accounts. The fundamental equation: **Assets = Liabilities + Equity**

Every transaction moves money FROM one account TO another. This makes the system self-balancing — any discrepancy shows up as a mismatch between debits and credits.

Example: Alice transfers \$100 to Bob

DEBIT Alice's account \$100 (money out of Alice)
CREDIT Bob's account \$100 (money into Bob)

Rules:

DEBIT an asset account = money going OUT of that account
CREDIT an asset account = money coming IN to that account
DEBIT = CREDIT always for a balanced transaction

Ledger entries:

entry_id	txn_id	account_id	type	amount
1	T001	Alice	DEBIT	100.00
2	T001	Bob	CREDIT	100.00

Verification: $\text{sum}(\text{DEBIT}) = \text{sum}(\text{CREDIT}) = 100.00 \checkmark$

Balance calculation

```
-- NEVER store a running balance as a separate column (it becomes inconsistent).  
-- ALWAYS calculate balance from the ledger:
```

```
SELECT  
  account_id,  
  sum(CASE WHEN entry_type = 'CREDIT' THEN amount  
        WHEN entry_type = 'DEBIT' THEN -amount  
        END) AS current_balance  
FROM ledger_entries  
WHERE account_id = 42  
AND status = 'posted';
```

Complete DDL Script

```
-- =====  
-- BANKING SYSTEM SCHEMA  
-- =====  
  
-- =====  
-- CUSTOMERS (KYC)  
-- =====  
  
CREATE TABLE customers (  
  customer_id      BIGINT      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  customer_number  VARCHAR(20) NOT NULL UNIQUE,  
  first_name       VARCHAR(100) NOT NULL,  
  last_name        VARCHAR(100) NOT NULL,  
  email            VARCHAR(255) NOT NULL UNIQUE
```

```

        CHECK (email ~ '^[^@\\s]+@[^@\\s]+\\.([^@\\s]+)$'),
phone          VARCHAR(20),
date_of_birth  DATE          NOT NULL,
ssn_hash       VARCHAR(100),  -- hashed SSN for KYC (NEVER store plain)
kyc_status     TEXT          NOT NULL DEFAULT 'pending'
        CHECK (kyc_status IN
('pending','in_review','verified','rejected')),
kyc_verified_at TIMESTAMPTZ,
address_line1  TEXT,
city           TEXT,
state_code     CHAR(2),
postal_code    VARCHAR(20),
country_code   CHAR(2)       NOT NULL DEFAULT 'US',
is_active      BOOLEAN       NOT NULL DEFAULT TRUE,
created_at     TIMESTAMPTZ   NOT NULL DEFAULT now(),
updated_at     TIMESTAMPTZ   NOT NULL DEFAULT now()
);

CREATE INDEX idx_customers_email ON customers (email);
CREATE INDEX idx_customers_number ON customers (customer_number);

COMMENT ON COLUMN customers.ssn_hash IS 'SHA-256 hash of SSN for KYC matching. Never
store raw SSN.';

-- =====
-- ACCOUNT TYPES
-- =====

CREATE TABLE account_types (
    account_type_id SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    code             VARCHAR(20) NOT NULL UNIQUE,
    name             VARCHAR(100) NOT NULL,
    description       TEXT,
    allows_overdraft BOOLEAN     NOT NULL DEFAULT FALSE,
    overdraft_limit  NUMERIC(15,4) NOT NULL DEFAULT 0,
    interest_rate    NUMERIC(8,6) NOT NULL DEFAULT 0 -- annual rate, e.g. 0.02 =
2%
        CHECK (interest_rate >= 0),
    minimum_balance NUMERIC(15,4) NOT NULL DEFAULT 0,
    is_active        BOOLEAN     NOT NULL DEFAULT TRUE
);

INSERT INTO account_types (code, name, allows_overdraft, interest_rate) VALUES
('CHK', 'Checking Account', TRUE, 0.0000),
('SAV', 'Savings Account', FALSE, 0.0200),
('LOA', 'Loan Account', FALSE, 0.0500),
('CRD', 'Credit Account', TRUE, 0.1900);

-- =====
-- ACCOUNTS
-- =====

```

```

CREATE TABLE accounts (
    account_id          BIGINT          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    account_number      VARCHAR(20)     NOT NULL UNIQUE,
    customer_id         BIGINT          NOT NULL REFERENCES customers(customer_id),
    account_type_id     SMALLINT        NOT NULL REFERENCES
account_types(account_type_id),
    currency_code       CHAR(3)         NOT NULL DEFAULT 'USD',
    status              TEXT            NOT NULL DEFAULT 'active'
        CHECK (status IN
('active','frozen','closed','dormant')),
    opened_at          TIMESTAMPTZ     NOT NULL DEFAULT now(),
    closed_at          TIMESTAMPTZ,
    -- Denormalized balance (maintained via triggers for performance)
    -- The ledger is the source of truth; this is a cache
    cached_balance     NUMERIC(19,4)   NOT NULL DEFAULT 0,
    cached_balance_at  TIMESTAMPTZ     NOT NULL DEFAULT now(),
    CONSTRAINT active_cant_have_closed_date CHECK (
        status != 'active' OR closed_at IS NULL
    )
);

CREATE INDEX idx_accounts_customer ON accounts (customer_id);
CREATE INDEX idx_accounts_number   ON accounts (account_number);
CREATE INDEX idx_accounts_status   ON accounts (status) WHERE status = 'active';

COMMENT ON COLUMN accounts.cached_balance IS
    'Cached for quick display. Source of truth is ledger_entries. Reconcile with
compute_balance().';

-- Function to compute true balance from ledger
CREATE OR REPLACE FUNCTION compute_balance(p_account_id BIGINT)
RETURNS NUMERIC(19,4)
LANGUAGE SQL STABLE AS $$
    SELECT COALESCE(
        sum(CASE WHEN entry_type = 'CREDIT' THEN amount
                WHEN entry_type = 'DEBIT'   THEN -amount END),
        0
    )
    FROM ledger_entries
    WHERE account_id = p_account_id
        AND status = 'posted';
$$;

-- =====
-- TRANSACTIONS
-- =====

CREATE TABLE transactions (
    txn_id              BIGINT          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    txn_reference       VARCHAR(50)     NOT NULL UNIQUE,
    txn_type            TEXT            NOT NULL
        CHECK (txn_type IN ('transfer','deposit','withdrawal',

```

```

'fee','interest','adjustment','reversal')),
    from_account_id    BIGINT          REFERENCES accounts(account_id),
    to_account_id      BIGINT          REFERENCES accounts(account_id),
    amount             NUMERIC(19,4)  NOT NULL CHECK (amount > 0),
    currency_code      CHAR(3)        NOT NULL DEFAULT 'USD',
    status             TEXT           NOT NULL DEFAULT 'pending'
                                CHECK (status IN
('pending','processing','completed','failed','reversed')),
    description        TEXT,
    initiated_by       BIGINT          REFERENCES customers(customer_id),
    initiated_at       TIMESTAMPTZ    NOT NULL DEFAULT now(),
    processed_at       TIMESTAMPTZ,
    -- External reference (wire transfer ID, ACH trace, etc.)
    external_reference VARCHAR(100),
    -- Risk and compliance
    ip_address         INET,
    is_suspicious      BOOLEAN        NOT NULL DEFAULT FALSE,
    CONSTRAINT transfer_has_both_accounts CHECK (
        txn_type != 'transfer' OR (from_account_id IS NOT NULL AND to_account_id IS
NOT NULL)
    ),
    CONSTRAINT different_accounts CHECK (from_account_id != to_account_id)
);

CREATE INDEX idx_txn_from_account ON transactions (from_account_id);
CREATE INDEX idx_txn_to_account  ON transactions (to_account_id);
CREATE INDEX idx_txn_reference   ON transactions (txn_reference);
CREATE INDEX idx_txn_status_date ON transactions (status, initiated_at DESC);

-- =====
-- LEDGER ENTRIES (Double-Entry)
-- =====

CREATE TABLE ledger_entries (
    entry_id          BIGINT          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    txn_id            BIGINT          NOT NULL REFERENCES transactions(txn_id),
    account_id        BIGINT          NOT NULL REFERENCES accounts(account_id),
    entry_type        CHAR(2)         NOT NULL CHECK (entry_type IN ('DR', 'CR')),
    amount            NUMERIC(19,4)  NOT NULL CHECK (amount > 0), -- always
positive!
    currency_code     CHAR(3)         NOT NULL DEFAULT 'USD',
    status            TEXT            NOT NULL DEFAULT 'posted'
                                CHECK (status IN ('pending','posted','reversed')),
    -- Running balance snapshot at this entry (denormalized for performance)
    running_balance   NUMERIC(19,4)  NOT NULL,
    description       TEXT,
    posted_at         TIMESTAMPTZ    NOT NULL DEFAULT now(),
    value_date        DATE            NOT NULL DEFAULT CURRENT_DATE -- effective date
for interest
);

```

```

CREATE INDEX idx_ledger_account_date ON ledger_entries (account_id, posted_at DESC);
CREATE INDEX idx_ledger_txn          ON ledger_entries (txn_id);
CREATE INDEX idx_ledger_value_date   ON ledger_entries (value_date);

-- =====
-- BALANCE INTEGRITY VERIFICATION
-- =====

-- View: verify double-entry balance for each transaction
CREATE VIEW transaction_balance_check AS
SELECT
    txn_id,
    sum(CASE WHEN entry_type = 'DR' THEN amount ELSE 0 END) AS total_debits,
    sum(CASE WHEN entry_type = 'CR' THEN amount ELSE 0 END) AS total_credits,
    sum(CASE WHEN entry_type = 'DR' THEN amount ELSE -amount END) AS net_balance,
    sum(CASE WHEN entry_type = 'DR' THEN amount ELSE -amount END) = 0 AS is_balanced
FROM ledger_entries
WHERE status = 'posted'
GROUP BY txn_id;

-- Alert: find any unbalanced transactions
CREATE VIEW unbalanced_transactions AS
SELECT * FROM transaction_balance_check WHERE NOT is_balanced;

-- =====
-- TRANSFER FUNCTION (Atomic)
-- =====

CREATE OR REPLACE FUNCTION execute_transfer(
    p_from_account_id  BIGINT,
    p_to_account_id    BIGINT,
    p_amount            NUMERIC(19,4),
    p_description       TEXT DEFAULT NULL,
    p_reference         TEXT DEFAULT NULL
)
RETURNS BIGINT -- returns txn_id
LANGUAGE plpgsql AS $$
DECLARE
    v_txn_id          BIGINT;
    v_from_balance     NUMERIC(19,4);
    v_to_balance       NUMERIC(19,4);
    v_from_type        account_types%ROWTYPE;
    v_ref              TEXT;
BEGIN
    -- Validate amount
    IF p_amount <= 0 THEN
        RAISE EXCEPTION 'Transfer amount must be positive: %', p_amount;
    END IF;

    -- Lock accounts in consistent order (lower ID first to prevent deadlock)
    IF p_from_account_id < p_to_account_id THEN
        PERFORM 1 FROM accounts WHERE account_id = p_from_account_id FOR UPDATE;

```

```

        PERFORM 1 FROM accounts WHERE account_id = p_to_account_id FOR UPDATE;
ELSE
    PERFORM 1 FROM accounts WHERE account_id = p_to_account_id FOR UPDATE;
    PERFORM 1 FROM accounts WHERE account_id = p_from_account_id FOR UPDATE;
END IF;

-- Check from account status and balance
SELECT at.*
INTO v_from_type
FROM accounts a
JOIN account_types at ON at.account_type_id = a.account_type_id
WHERE a.account_id = p_from_account_id AND a.status = 'active';

IF NOT FOUND THEN
    RAISE EXCEPTION 'Source account % is not active', p_from_account_id;
END IF;

v_from_balance := compute_balance(p_from_account_id);

IF NOT v_from_type.allows_overdraft AND v_from_balance < p_amount THEN
    RAISE EXCEPTION 'Insufficient funds. Balance: %, Required: %',
        v_from_balance, p_amount;
END IF;

-- Generate unique reference if not provided
v_ref := COALESCE(p_reference,
    'TXN-' || to_char(now(), 'YYYYMMDD-HH24MISS') || '-' ||
gen_random_uuid()::TEXT);

-- Create transaction record
INSERT INTO transactions (txn_reference, txn_type, from_account_id,
to_account_id,
                        amount, status, description, initiated_at,
processed_at)
VALUES (v_ref, 'transfer', p_from_account_id, p_to_account_id,
        p_amount, 'completed', p_description, now(), now())
RETURNING txn_id INTO v_txn_id;

-- Get balances for running_balance calculation
v_from_balance := compute_balance(p_from_account_id);
v_to_balance   := compute_balance(p_to_account_id);

-- Create DEBIT entry for source account
INSERT INTO ledger_entries
    (txn_id, account_id, entry_type, amount, running_balance, description,
posted_at)
VALUES
    (v_txn_id, p_from_account_id, 'DR', p_amount,
        v_from_balance - p_amount, p_description, now());

-- Create CREDIT entry for destination account
INSERT INTO ledger_entries

```

```

        (txn_id, account_id, entry_type, amount, running_balance, description,
        posted_at)
    VALUES
        (v_txn_id, p_to_account_id, 'CR', p_amount,
         v_to_balance + p_amount, p_description, now());

    -- Update cached balances
    UPDATE accounts SET
        cached_balance = v_from_balance - p_amount,
        cached_balance_at = now()
    WHERE account_id = p_from_account_id;

    UPDATE accounts SET
        cached_balance = v_to_balance + p_amount,
        cached_balance_at = now()
    WHERE account_id = p_to_account_id;

    RETURN v_txn_id;
END;
$$;

-- =====
-- STATEMENT / HISTORY
-- =====

CREATE TABLE account_statements (
    statement_id    BIGINT      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    account_id      BIGINT      NOT NULL REFERENCES accounts(account_id),
    period_start    DATE        NOT NULL,
    period_end      DATE        NOT NULL,
    opening_balance NUMERIC(19,4) NOT NULL,
    closing_balance NUMERIC(19,4) NOT NULL,
    total_credits   NUMERIC(19,4) NOT NULL,
    total_debits    NUMERIC(19,4) NOT NULL,
    generated_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT valid_period CHECK (period_end > period_start)
);

CREATE INDEX idx_statements_account ON account_statements (account_id, period_end
DESC);

```

Key Design Decisions

1. **Double-entry ledger** — Every financial event creates two entries. The system is self-verifying.
2. **Amount always positive in ledger** — The sign is carried by `entry_type` (DR/CR). No negative numbers in the `amount` column. Simpler validation, clearer intent.
3. **Balance computed from ledger** — `compute_balance()` is the authoritative source. `cached_balance` is a performance optimization that must be periodically reconciled.

4. **Lock ordering in transfers** — Always lock accounts in ascending `account_id` order to prevent deadlocks in concurrent transfers.
 5. **Atomic transfer function** — The entire transfer (transaction record + both ledger entries + balance update) is one database transaction. Either all succeed or none do.
 6. **Running balance in ledger** — A snapshot of the balance after each entry. Allows fast statement generation without re-scanning all history.
-

Common Queries

```
-- Account statement: last 30 entries
SELECT
    le.posted_at,
    t.txn_type,
    t.description,
    CASE le.entry_type WHEN 'CR' THEN le.amount ELSE NULL END AS credit,
    CASE le.entry_type WHEN 'DR' THEN le.amount ELSE NULL END AS debit,
    le.running_balance AS balance
FROM ledger_entries le
JOIN transactions t ON t.txn_id = le.txn_id
WHERE le.account_id = 1001
    AND le.status = 'posted'
ORDER BY le.posted_at DESC
LIMIT 30;

-- Verify balance integrity (should return 0 rows)
SELECT * FROM unbalanced_transactions;

-- All transactions for a customer
SELECT
    t.txn_reference,
    t.txn_type,
    a_from.account_number AS from_account,
    a_to.account_number AS to_account,
    t.amount,
    t.status,
    t.initiated_at
FROM transactions t
LEFT JOIN accounts a_from ON a_from.account_id = t.from_account_id
LEFT JOIN accounts a_to ON a_to.account_id = t.to_account_id
WHERE a_from.customer_id = 42 OR a_to.customer_id = 42
ORDER BY t.initiated_at DESC;

-- Daily transaction volume
SELECT
    date_trunc('day', initiated_at)::DATE AS txn_date,
    count(*) AS txn_count,
    sum(amount) AS total_volume
FROM transactions
```

```
WHERE status = 'completed'
      AND initiated_at >= now() - INTERVAL '30 days'
GROUP BY 1
ORDER BY 1;
```

ACID Guarantees in Banking

ATOMICITY:	The transfer function wraps everything in one transaction. If any step fails (insufficient funds, account frozen, DB error), the entire transfer is rolled back – no partial transfers.
CONSISTENCY:	CHECK constraints, NOT NULL, and application logic ensure the schema is always in a valid state. Double-entry verification catches balance mismatches.
ISOLATION:	Row-level locks on accounts during transfer prevent two concurrent transfers from seeing each other's intermediate state. PostgreSQL's SERIALIZABLE isolation level provides the strongest guarantee.
DURABILITY:	PostgreSQL's WAL (Write-Ahead Log) ensures committed transactions survive crashes.

Common Mistakes

1. **Storing balance as a mutable column** — If `balance` is updated directly, concurrent transactions can create race conditions that lose money.
2. **Single-entry recording** — Recording only one side of a transaction. Double-entry provides built-in error detection.
3. **Allowing negative amounts in ledger** — Use `entry_type` for direction; keep `amount` always positive. Simplifies validation and reporting.
4. **Not locking accounts before checking balance** — Without `SELECT FOR UPDATE`, two concurrent transfers can both succeed when only one should (TOCTOU race condition).
5. **Using FLOAT for money** — Rounding errors accumulate in financial calculations. Always use `NUMERIC`.

Best Practices

1. **Use NUMERIC(19,4)** for all monetary amounts.
2. **Always use the transfer function** — never insert ledger entries manually outside of a controlled function.
3. **Verify double-entry balance** periodically (automated reconciliation job).
4. **Lock in a consistent order** to prevent deadlocks.
5. **Never delete ledger entries** — use reversal entries instead.
6. **Audit everything** — every account change, balance query, and transfer should be logged.

Performance Considerations

```
-- Balance calculation on large ledger tables
-- Partition ledger_entries by year for fast recent-data access
CREATE TABLE ledger_entries_2024
    PARTITION OF ledger_entries
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');

-- BRIN index on posted_at (sequential data)
CREATE INDEX idx_ledger_brin ON ledger_entries USING BRIN (posted_at);

-- For statement generation: covering index
CREATE INDEX idx_ledger_account_covering ON ledger_entries
    (account_id, posted_at DESC)
    INCLUDE (entry_type, amount, running_balance, description);
-- Allows index-only scans for statement queries!
```

Interview Questions & Answers

Q1: Why use double-entry bookkeeping in a database?

A: Double-entry provides built-in error detection. Every transaction must balance (debits = credits). If money disappears or appears unexpectedly, the imbalance is immediately detectable. It also provides a complete audit trail — every account change is recorded as a ledger entry with the corresponding transaction.

Q2: Why should balance not be stored directly as a column and updated on each transaction?

A: Because concurrent transactions create race conditions. Two transactions reading the same balance and both updating it will produce an incorrect result — one update will be lost. The safe approach: compute balance from the immutable ledger. For performance, cache the balance but reconcile it from the ledger periodically.

Q3: How do you prevent deadlocks in fund transfers?

A: Always lock accounts in a consistent order — for example, always lock the lower `account_id` first. If transaction A locks accounts (1, 2) and transaction B locks accounts (2, 1) in that order, they can deadlock. Locking in ID order ensures A locks 1 then 2, and B also locks 1 then 2 — B waits for A, no deadlock.

Q4: What is a reversal entry and why should you use it instead of deletion?

A: A reversal entry creates new ledger entries with opposite debit/credit signs to cancel the effect of a previous entry, while preserving the audit trail. Deleting ledger entries would break the audit trail and potentially cause balance discrepancies. Regulators require that all financial transactions be permanently recorded.

Q5: How do you calculate the balance for an account at a specific point in time?

A: Sum all posted ledger entries up to that timestamp: `WHERE account_id = X AND posted_at <= '2024-01-31' AND status = 'posted'`. If running balances are stored in ledger entries, you can also just look up the last entry before that timestamp.

Exercises with Solutions

Exercise 1

Write a reconciliation query that compares `accounts.cached_balance` to the computed balance from the ledger and identifies any accounts with discrepancies.

Solution:

```
SELECT
    a.account_id,
    a.account_number,
    a.cached_balance,
    compute_balance(a.account_id) AS ledger_balance,
    a.cached_balance - compute_balance(a.account_id) AS discrepancy
FROM accounts a
WHERE a.status = 'active'
      AND abs(a.cached_balance - compute_balance(a.account_id)) > 0.001
ORDER BY abs(a.cached_balance - compute_balance(a.account_id)) DESC;
```

Exercise 2

Use the transfer function to perform a \$500 transfer from account 1001 to account 1002.

Solution:

```
BEGIN;
SELECT execute_transfer(
    1001,          -- from account
    1002,          -- to account
    500.00,        -- amount
    'Monthly rent payment', -- description
    'RENT-2024-01' -- reference
);
COMMIT;
-- Verify:
SELECT compute_balance(1001) AS alice_balance, compute_balance(1002) AS bob_balance;
```

Cross-References

- `05_schema_design_ecommerce.md` — payment integration
- `10_design_patterns.md` — audit trails
- `../09_Transactions_Concurrency/` — ACID properties, row locking
- `../05_PostgreSQL_Core/01_numeric_types.md` — NUMERIC for money